

Reviewer Pack — Algebraic Geometry over Finite Fields and EF Proof Length

Entry 797387c6c6e5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-30 23:07:03 UTC

Algebraic Geometry over Finite Fields and EF Proof Length

Entry ID: 797387c6c6e5 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-30 23:07:03 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry over Finite Fields **Field B** (complexity object): Extended Frege Proof Length

Statement:

For any CNF tautology derived from a system of polynomial equations defining a variety over $\text{GF}(2^k)$, the Extended Frege proof length is super-polynomial in the number of variables n .

Rationale (proposer's reasoning):

The algebraic structure of varieties over finite fields imposes constraints that may require intricate proof steps, which Extended Frege systems lack efficient tools to handle, suggesting inherent complexity in proof derivation.

Taxonomy category: PROOF_COMPLEXITY_EXT_FREGE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 81c0694fc2c2175f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.80	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True

def generate_primes(count):
    primes = []
    num = 2
    while len(primes) < count:
        if is_prime(num):
            primes.append(num)
        num += 1
    return primes

def extended_gcd(a, b):
    if a == 0:
        return (b, 0, 1)
    gcd, x1, y1 = extended_gcd(b % a, a)
    x = y1 - (b // a) * x1
    y = x1
    return (gcd, x, y)

def mod_inverse(a, m):
    gcd, x, _ = extended_gcd(a, m)
    if gcd != 1:
        raise ValueError("Inverse doesn't exist")
    else:
        return x % m

def polynomial_to_cnf(poly, n):
    cnf = []
    for term in poly:
        clause = [term[i] * (-1) ** (i % 2) for i in range(n)]
        cnf.append(clause)
    return cnf

def dpll_search_tree_size(cnf):
    def dpll(clauses, assignment):
```

```

        if not clauses:
            return 0
        if any(all(lit in assignment and assignment[lit] == val for lit, val in clause) for clause in clauses):
            return 0
        literals = [lit for clause in clauses for lit in clause if lit not in assignment]
        literal = literals[0]
        new_clauses = []
        for clause in clauses:
            if literal in clause:
                new_clauses.append([l for l in clause if l != literal])
            elif -literal in clause:
                continue
            else:
                new_clauses.append(clause + [-literal])
        return 1 + max(dpll(new_clauses, assignment + {literal: True}), dpll(new_clauses, assignment + {-literal: True}))

    return dpll(cnf, {})

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    k = random.randint(1, 3)
    GF = [i for i in range(2 ** k)]
    poly = [random.choice(GF) for _ in range(n)]
    cnf = polynomial_to_cnf(poly, n)
    depth = dpll_search_tree_size(cnf)
    instances_tested = 1
    conjecture_holds = depth > n * math.log(n)
    counterexample = "" if conjecture_holds else "mapping_undefined"
    return {
        "metric_name": "DPLL Search Tree Depth",
        "metric_value": depth,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or generate_primes(30)
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_depth = sum(r["metric_value"] for r in results) / len(results)
    std_depth = math.sqrt(sum((r["metric_value"] - mean_depth) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_depth} std={std_depth} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
    else:

```

```
print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0cc0ddc8.py", line 100, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0cc0ddc8.py", line 82, in run_trial
    cnf = polynomial_to_cnf(poly, n)
        ^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0cc0ddc8.py", line 52, in polynomial_to_cnf
    clause = [term[i] * (-1) ** (i % 2) for i in range(n)]
             ~~~~^^^
```

TypeError: 'int' object is not subscriptable

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with TypeError due to subscripting an integer, preventing data collection
| next: Fix the polynomial_to_cnf function's clause generation logic to handle integer indexing

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	89724
2	preregistration	ollama_remote	qwen3:8b	0	0	24074
3	novelty	ollama_remote	qwen3:8b	0	0	20762
4	novelty	ollama_remote	qwen3:8b	0	0	14730
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13433
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10466
7	judge	ollama_remote	qwen3:8b	0	0	15087

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 188276 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in research/audit/797387c6c6e5.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/797387c6c6e5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/797387c6c6e5.tar.gz` (if generated)